

Description MultiSigWallet.sol

Overview of the Contract

This smart contract implements a multi-signature wallet (MultiSig Wallet). The purpose of wallet is that a transaction requires confirmation from multiple owners, more than just one person.

Vars

Ownetsd massive

```
address[] public owners;
```

Stores a list of all wallet owners

Owner verification

```
mapping(address => bool) public isOwner;
```

Allows you to quickly check whether the address belongs to the owner.

For exmaple:

```
isOwner[msg.sender]
```

It sends **true** - owner has access.

Amount of aproves

```
uint public required;
```

Shows how many aprovals need for transaction. Minimum 2 transactions.

Structure Transaction

```
struct Transaction {  
    address to;  
    uint value;  
    bytes data;  
    bool executed;
```

```
    uint numConfirmations;  
}
```

Transaction information

Vars:

to

Receiver addr

value

Amount ETH to send

data

additional data for other contraction functions.
if sends only eth it could be void.

executed

if this transaction already complited.
It pupose for only one operation.

numConfirmations

Num confirmations from ovner

Transaction storage

```
Transaction[] public transactions;
```

massive of all transactions

every transaction has their own index: **0, 1, 2, etc...**

transaction confirmation

```
mapping(uint => mapping(address => bool)) public confirmations;
```

shows, which owner approved this transaction.

exmple:

```
confirmations[0][owner]
```

if **true**, owner approved transaction with index 0.

Events

need to debug informtaion to blockchain logs.

Deposit

```
event Deposit(address indexed sender, uint amount);
```

triggers when done eth contract

Submit

```
event Submit(uint indexed txIndex);
```

creation new transaction

Confirm

```
event Confirm(address indexed owner, uint indexed txIndex);
```

approves transaction by owner

Execute

```
event Execute(uint indexed txIndex);
```

transaction execution

Revoke

```
event Revoke(address indexed owner, uint indexed txIndex);
```

cancels approval

Modifiers

Modificatios uses for checking conditions before execution function

onlyOwner

```
modifier onlyOwner()
```

Allows only wallet owners to call the function.

txExists

```
modifier txExists(uint _txIndex)
```

Checks if transaction exists or not.

notExecuted

```
modifier notExecuted(uint _txIndex)
```

Checks if transaction not executed

notConfirmed

```
modifier notConfirmed(uint _txIndex)
```

chekcs if user not confirmed transaction

Constructor

```
constructor(address[] memory _owners, uint _required)
```

executes at contract creation.

What is doing:

1. check if list is not void
2. check of amount confirmed are correct
3. add owners to massive
4. dublication check
5. saves minimal amount of approvals

receive()

```
receive() external payable
```

function for receiveing eth on contract.
when smeone sends eth:

- contract reseives eth
- creates event `deposit`

`submitTransaction()`

```
function submitTransaction(...)
```

new transaction creation

owner gives:

- receiver address
- eth amount
- transaction data

var of this transaction saves to massive

`confirmTransaction()`

```
function confirmTransaction(uint _txIndex)
```

transaction approval from owner

there is:

1. checks if trqansaction exists
2. check if transaction not executed
3. checks if ownr not aprooved
4. approval amount increments
5. saves approval info

`executeTransaction()`

```
function executeTransaction(uint _txIndex)
```

execute transaciton

before exeecuting checks if:

```
txn.numConfirmations >= required
```

is enough confirmations

if enough:

- transaction saves like executed
- ETH sends to receiver `.call()`.

`revokeConfirmation()`

```
function revokeConfirmation(uint _txIndex)
```

cancel transaction by owner

there is:

- approval decrement
- info about approval removes

`getTransactionCount()`

```
function getTransactionCount()
```

returns all transactio

Example how it works

there is:

- 3 owners
- needs 2 approvals minimum

1

Contract deploys

2

on contract send eth

3

one of owners creates transactio

- who receiver

- how much eth

4

other owners approves

5

when approvals amount is enough, any owner can execute transaciton

Conclusion

main buisness logic:

- several owners manages wallet
- first of all transactions creates
- then transactions appoves by other owners
- when approvals amount is enough, any owner can execute transaciton

it gives:

- securelly store ETH
- manage wallet by several owners
- reduce the risk of losing money